一、优化 *03:08*

1. Vue优化 03:47

-

- **列表渲染优化**：对于通过循环生成的列表，应给每个列表项一个稳定且唯一的key，这有利于在列表变动时尽量减少DOM操作（删除、新增、改动元素）
- **条件渲染选择**：使用v-show替代v-if，因为v-show只是切换CSS的display属性，而v-if会销毁和重建组件
- **延迟加载**：使用延迟装载(defer)技术来优化初始加载性能
- **对象冻结**：冻结的对象不会被响应化，可以减少Vue的响应式处理开销
- **函数式组件**：对于无状态组件，可以使用函数式组件来提高性能
- **计算属性缓存**：如果模板中某个数据会使用多次，并且该数据是通过计算得到的，应该使用计算属性以缓存计算结果

1）优化原则

- **避免过早优化**：优化会带来开发时间延长、代码可读性降低等代价，应在真正需要时再进行优化
- **性能问题导向**：只有当程序出现明显性能问题时才需要进行针对性优化
- **因地制宜**：优化没有固定模式，需要根据具体场景选择合适的优化手段
- **思想重于技巧**：掌握底层原理和编程思想比记忆具体优化技巧更重要

2）优化代价

- **开发成本**：优化会增加开发时间和人力成本
- **可维护性**：过度优化可能导致代码难以阅读和维护
- **边际效应**：微小的性能提升可能对用户体验没有实际影响

3）优化方法论

- **问题分析**：首先需要准确识别性能瓶颈所在
- **针对性解决**：根据具体问题选择合适的优化手段
- **权衡取舍**：在性能和可维护性之间找到平衡点

持续演进：随着需求变化不断调整优化策略

- 注：笔记中省略了课程中关于编程哲学和教学引导的内容，仅保留了具体的Vue优化知识点。

2. vue常见优化手段 06:23

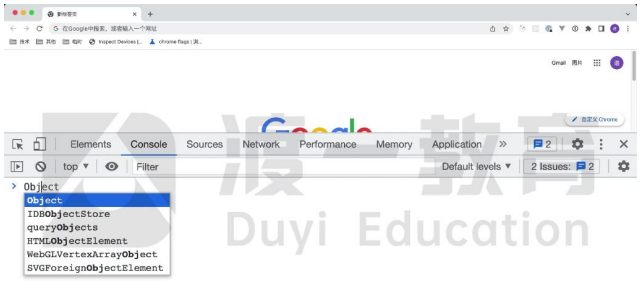
1）使用key 07:33

- **稳定且唯一原则**：在循环生成的列表中，每个列表项需要设置稳定且唯一的key值，有利于列表变动时最小化DOM操作（删除/新增/改动）

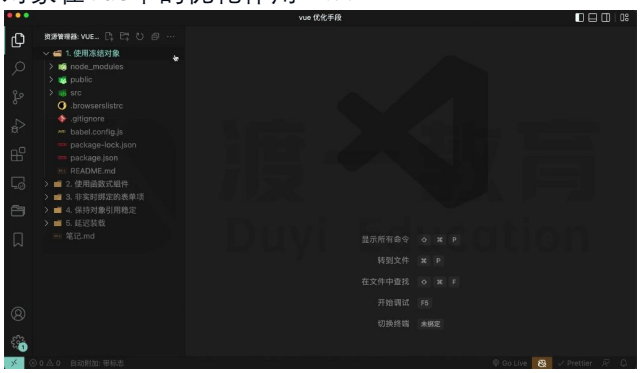
- **下标不适用原因**：虽然数组下标具有唯一性，但不够稳定（如插入新元素会导致后续元素下标全部变化）
- **正确示例**：使用数据项的ID作为key值（如用户ID），既保证唯一性又保持稳定性
- **错误示例**：用数组索引作为key值，在数据顺序变化时会导致不必要的DOM重建

2）使用冻结的对象 09:09

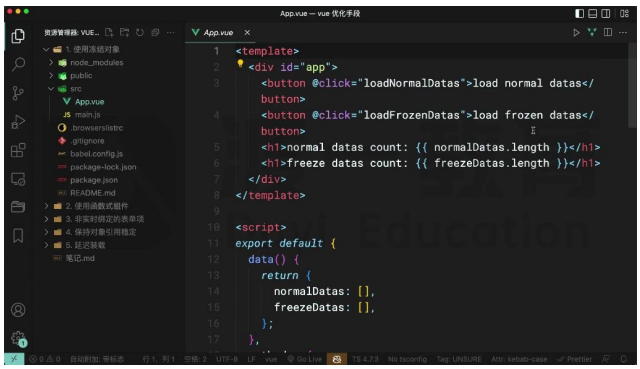
- 冻结对象的定义与特性 13:13



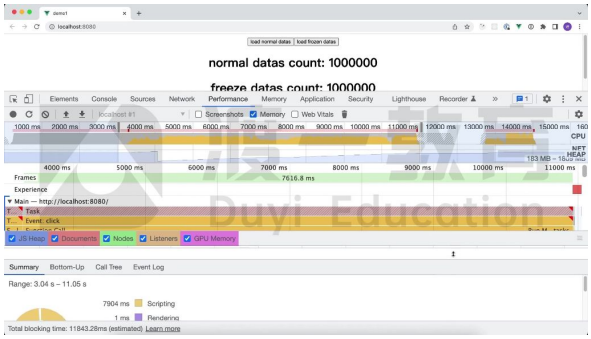
  -
  - **定义**：通过Object.freeze()方法使对象不可修改（不能添加/删除/修改属性）
  - **检测方法**：使用Object.isFrozen()判断对象是否被冻结
  - **特性验证**：
    - 修改属性值无效（如obj.a=3不会生效）
    - 添加新属性无效（如obj.b=5不会生效）
- 冻结对象在Vue中的优化作用 14:41



  -
  - **响应式原理**：Vue通过递归遍历对象属性，将其转换为getter/setter实现响应式
  - **性能消耗点**：深度嵌套的大对象（如商品列表）的响应式转换需要较长时间
  - **优化机制**：当Vue检测到冻结对象时，会跳过响应式处理流程
  - **适用场景**：不需要修改的静态数据（如商品展示列表、新闻列表等）
- 示例：冻结对象与普通对象的性能对比 20:17



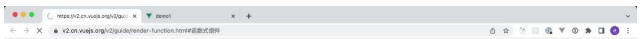
  -
  - **测试方案**：
    - 生成包含100万个嵌套对象的数组

        - 对比普通数组赋值与冻结数组赋值的性能差异
    - **关键代码**：
    - **性能分析**：
        - 普通数据加载：耗时约7秒（主要消耗在响应式转换）
        - 冻结数据加载：耗时约1.3秒（跳过响应式处理）
        - 性能提升：约5倍速度提升
        - 

    - **Chrome Performance工具分析**：
        - 普通数据：出现大量"Proxy setter"调用（响应式处理）
        - 冻结数据：仅见数据生成过程，无响应式处理开销
    - **内存观察**：
        - 普通数据：带有__ob__观察者标记
        - 冻结数据：保持原始对象结构
    - **优化建议**：
        - 对于纯展示的大数据列表优先使用冻结
        - 需要响应式更新的数据保持普通状态
        - 避免过度优化，根据实际性能瓶颈针对性处理

3）使用函数式组件 28:43

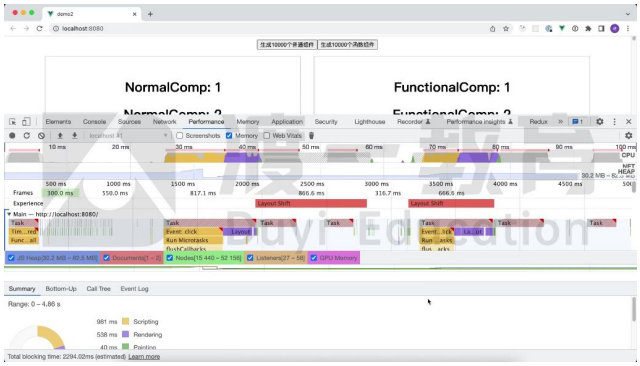
- 函数式组件的概念 28:48



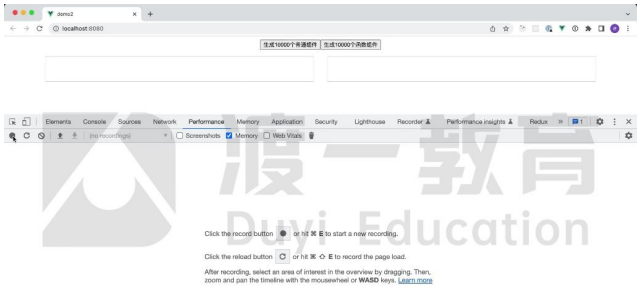
渡一教育
Duyi Education

    -
    - **定义**：函数式组件是通过设置functional: true配置的纯渲染组件，没有实例和响应式数据
    - **官方说明**：需要参考Vue.js官方文档中关于渲染函数和函数式组件的章节
    - **类比说明**：类似于React早期的函数式组件，但功能相对简单
- 函数式组件的特点 29:22

-

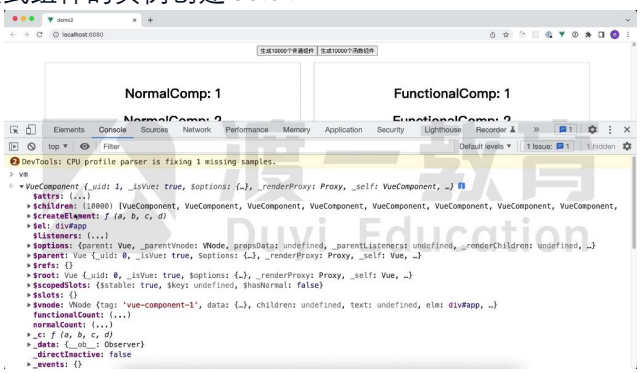
  - **无状态性**：没有响应式数据（data）和实例（this上下文）
  - **配置方式**：必须在组件配置中添加functional: true选项
  - **props处理**：2.3.0+版本可省略props选项，所有attribute自动解析为prop
  - **纯渲染特性**：仅接收props并渲染界面，不做状态管理
- 函数式组件与普通组件的对比 31:05
  - **代码结构**：
    - 普通组件：标准组件配置对象
    - 函数式组件：需添加functional配置
  - **功能差异**：
    - 普通组件：可管理状态、有生命周期
    - 函数式组件：仅做纯渲染
  - **实例创建**：
    - 普通组件：Vue会为每个组件创建实例
    - 函数式组件：不会创建组件实例
- 函数式组件的性能优势 34:17
  - 

  - **执行效率**：
    - 普通组件：脚本执行时间约538ms，渲染时间241ms
    - 函数式组件：脚本执行时间约515ms，渲染时间242ms
  - **优化原理**：减少实例创建和虚拟DOM操作的开销
  - **适用场景**：适合大量重复的简单展示型组件
- 函数式组件的内存消耗 34:36



-
- **内存增量**：
  - 普通组件：增加约30MB内存
  - 函数式组件：增加约23MB内存
- **优化效果**：内存占用减少约23%
- **扩展说明**：在10万/100万量级组件时优势更明显

- 函数式组件的实例创建 36:04



-
- **实例差异**：
  - 普通组件：会在$children中创建VueComponent实例
  - 函数式组件：不会出现在组件树中
- **虚拟DOM表现**：
  - 普通组件：显示为组件节点
  - 函数式组件：直接渲染为原生DOM节点（如h1）
- **性能关键**：减少实例创建是性能提升的主要原因

4）使用计算属性 38:26

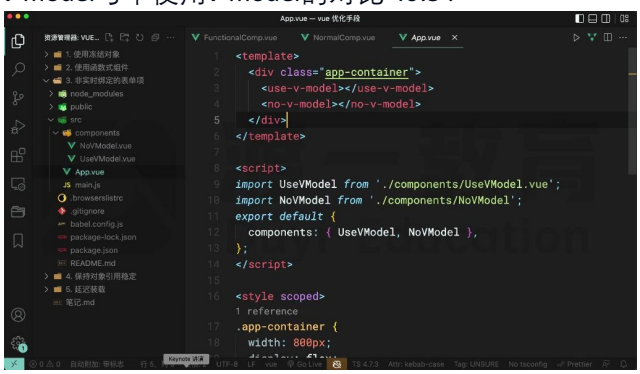
-
- **缓存机制**：当模板中某个数据会被多次使用且通过计算得到时，使用计算属性可以缓存计算结果，避免重复计算
- **响应式原理**：计算属性会自动追踪依赖，当依赖数据变化时会重新计算，否则直接返回缓存值

- **适用场景**：特别适合用于需要复杂计算的模板数据，如过滤后的列表、格式化后的日期等

5）非实时绑定的表单项 38:39

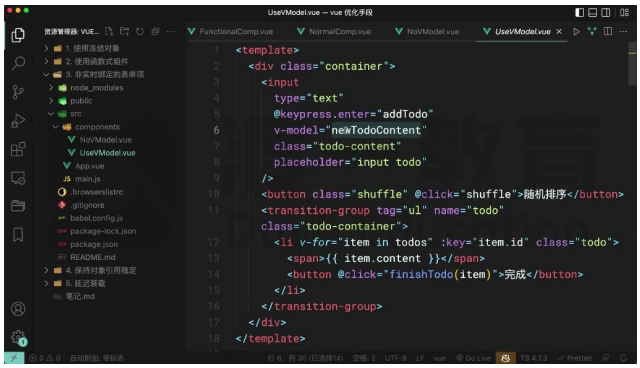
- v-model绑定表单项的性能问题 39:02
  -

  - **重渲染机制**：使用v-model时，每次输入都会触发数据变更，导致Vue重新渲染整个组件
  - **性能瓶颈**：在输入长文本（如"abcdefg"）时会产生多次渲染，若同时有其他JS操作会明显卡顿
  - **根本原因**：双向绑定机制强制保持数据和UI同步，每次输入都会触发完整的响应式更新流程
- 非实时绑定的解决方案 39:26
  -

  - **lazy修饰符**：使用v-model.lazy只在change事件（如失焦）时同步数据
  - **事件监听**：完全不用v-model，改为监听@keypress.enter等事件手动更新数据
  - **代价说明**：这些方案会导致数据与UI短暂不一致，需要权衡实时性和性能
- 使用v-model与不使用v-model的对比 40:34
  - 

  - **实验设计**：创建两个相同功能的TodoList组件，一个使用v-model，一个使用事件监听
  - **性能表现**：
    - v-model版本：输入时动画卡顿，因为主线程忙于处理频繁的重渲染

        - 非v-model版本：动画流畅，仅在回车时触发一次更新
    - **原理分析**：动画本应在合成线程运行，但v-model导致DOM操作阻塞了主线程
- v-model的优势与代价 43:21



    - **核心优势**：始终保持数据与UI同步，简化开发逻辑
    - **使用代价**：
        - 性能损耗：频繁触发响应式更新
        - 数据一致性：优化方案可能导致短暂不一致
    - **决策原则**：根据实际场景权衡，出现性能问题时再考虑优化方案

6）保持对象引用稳定 44:02

- **重渲染触发条件**:　Vue触发rerender的时机是其依赖的数据发生变化。若数据没有变化，即使重新赋值也不会处理。
- **数据变化判断**:　通过$hasChanged(x,y)$函数判断，当$x===y$时返回false表示无变化，否则返回true。
    - 特殊处理：对于0值，需额外判断$1/x!==1/y$（考虑+0和-0的情况）
    - NaN处理：通过$x===x||y===y$判断
- **稳定策略**:
    - 原始数据类型：保持值不变
    - 对象类型：保持引用不变
- **实现原理**: Vue通过对象引用比较决定是否重新渲染组件，引用不变则不触发虚拟DOM更新。

7）v-model修饰符与渲染优化

-

- **.lazy修饰符**:
  - 默认监听input事件，使用.lazy后改为监听change事件
  - 效果：减少触发频率，仅在失去焦点时同步数据
  - **注意**：会导致数据与表单项暂时不一致
- **替代方案**:
  - 不使用v-model，手动管理数据绑定
  - 使用计算属性进行中间处理
- **优化技巧**:
  - 用v-show替代v-if（保持DOM稳定）
  - 延迟装载(defer)延迟非关键内容渲染
  - 细分组件减少重渲染范围

8）Vue源码解析

-

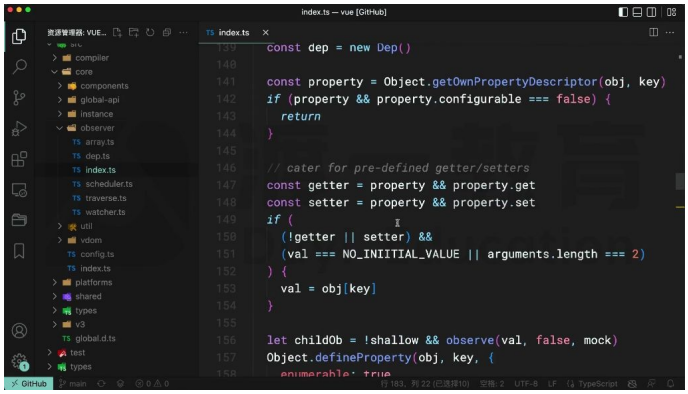
- **核心函数**:

```
function hasChanged(x, y) {
 if(x === y){
  return x === 0 && 1/x !== 1/y
 }else{
  return x === x || y === y
 }
}
```

- **特殊值处理**:
  - +0/-0区分：通过$1/x$的符号判断
  - NaN检测：利用$NaN! = = NaN$的特性
- **性能影响**:
  - 引用稳定的对象可跳过子组件diff
  - 频繁变动的数据应单独提取到最小组件

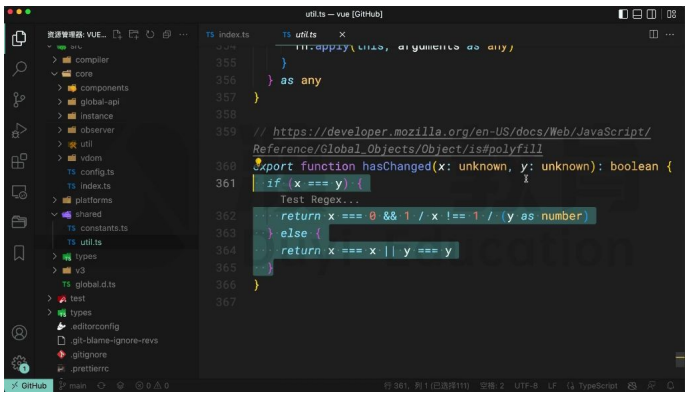
二、*Vue*数据响应式原理与优化 *46:05*

1. Vue数据变化通知机制 46:25

- 

- **Observer模式实现**：通过Object.defineProperty定义getter/setter，在属性访问时收集依赖(dep.depend())，在属性修改时触发通知(dep.notify())
- **深层监听处理**：通过childOb = !shallow & observe(val)实现嵌套对象的递归监听，确保对象属性变化也能触发更新
- **初始值处理逻辑**：当val为NO_INITIAL_VALUE且参数长度为2时，会取obj[key]作为初始值

2. Vue数据变化检测函数hasChanged 46:42

- 

- **基础判断逻辑**：
  - 当x === y时返回false，表示数据未变化
  - 当x !== y时进一步检查是否为NaN等特殊情况
- **特殊值处理**：
  - **NaN判断**：通过x = x || y = y排除NaN情况
  - **±0判断**：使用1/x !== 1/y区分+0和-0（1/+0=+∞，1/-0=-∞）
- **框架开发考量**：相比简单的x !== y判断，这种实现能正确处理JavaScript中所有原始值的边界情况

3. Vue优化手段与数据稳定性 47:01

-

- **关键优化原则**：

    - **引用稳定性**：对于对象类型，保持引用不变可避免子组件重渲染
    - **值稳定性**：对于原始类型，保持值不变可避免不必要的处理
- **具体优化手段**：
    - 使用v-show替代v-if保持DOM稳定
    - 使用计算属性缓存计算结果
    - 冻结不需要响应式的对象(Object.freeze)
    - 使用keep-alive缓存组件实例

4. Vue组件重渲染条件与性能优化 47:57

- **重渲染触发条件**：
    - 只有当依赖数据真实变化时才会触发rerender
    - 相同值的重复赋值不会引起重新渲染（如：1→1）
- **性能优化实践**：
    - **组件拆分**：将频繁变化的部分独立为子组件
    - **引用保持**：对于大型对象，修改属性而非替换整个对象
    - **计算属性**：将复杂计算移入computed避免重复计算

5. Vue特殊数据类型处理 48:51

- **NaN的特殊处理**：
    - NaN === NaN在JavaScript中返回false
    - Vue通过x === x检查排除NaN变化误判
- **±0的区分处理**：
    - +0 === -0返回true但实际是不同的值
    - 通过1/x !== 1/y准确识别±0的变化
- **框架开发启示**：必须考虑所有数据类型的边界情况，不能简单使用或=判断

6. Vue框架开发中的细节考虑 50:50

- **设计哲学**：
    - 框架级代码必须处理所有边界情况
    - 看似"无用"的基础知识（如IEEE754浮点数规范）在实际开发中至关重要
- **开发启示**：
    - 性能优化需要从数据变化检测的最底层做起
    - 简单的x !== y判断无法满足框架开发需求
    - 基础知识的深度决定代码质量上限

7. Vue组件属性引用稳定性示例 52:19

- **引用比较机制**：
    - Vue仅比较对象引用，不深度比较内容
    - 相同内容的不同对象实例会被判定为变化
- **实际影响**：
    - 替换整个对象会触发重渲染
    - 修改对象属性但保持引用不会触发重渲染
    - 优化时应尽量保持对象引用稳定

8. Vue组件重渲染次数统计示例 53:03

- **实现原理**：
    - 在beforeUpdate生命周期中递增计数器
    - 通过v-if控制计数器的显示
- **典型应用场景**：
    - 性能优化时验证组件更新频率

- 调试不必要渲染的问题
- 验证shouldComponentUpdate等效逻辑

三、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
冻结对象优化	使用Object.freeze()避免Vue对不需要响应式的数据进行深度遍历	**稳定且唯一**的key值 vs 仅唯一的key值	★★★
函数式组件	无状态组件，不创建Vue实例，减少内存占用和渲染时间	函数式组件无data vs 普通组件有data	★★
计算属性缓存	利用计算属性的缓存特性减少重复计算	计算属性缓存 vs 方法调用无缓存	★
v-model性能问题	避免过度使用v-model导致频繁重渲染，可使用.lazy修饰符或事件监听	v-model双向绑定 vs 事件监听单向数据流	★★★
对象引用稳定	保持对象引用稳定避免不必要的重新渲染	深比较 vs 浅比较（Vue默认）	★★★★
延迟装载	使用requestAnimationFrame分帧渲染大型组件	一次性渲染 vs 分批次渲染	★★★★
v-show替代v-if	频繁切换显示时使用v-show避免组件销毁重建	v-show（display切换） vs v-if（DOM操作）	★★
优化思想	**因地制宜、见招拆招**，避免过早优化带来的开发成本增加	问题驱动优化 vs 预防性优化	★★★★★
Vue响应式原理	通过Object.defineProperty实现数据劫持，冻结对象可跳过响应式处理	响应式对象 vs 冻结对象	★★★★
性能分析工具	使用浏览器Performance工具定位性能瓶颈	JS执行时间 vs 渲染时间分析	★★★